

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027



PR1 Introducción a Redis



Objetivo

Configurar un servidor **Redis 7** mediante **Docker**, establecer una conexión desde **JetBrains DataGrip** y realizar operaciones CRUD completas sobre estructuras clave-valor.

Al finalizar este laboratorio serás capaz de:

- Levantar Redis en un contenedor Docker con persistencia AOF.
- Verificar el estado del contenedor y la conectividad con Redis CLI.
- Crear una conexión en DataGrip y ejecutar comandos Redis desde la interfaz gráfica.
- Realizar operaciones CRUD (SET, GET, MSET, MGET, DEL, EXISTS).
- Gestionar expiración de claves (TTL, SETEX, EXPIRE).
- Utilizar contadores atómicos (INCR, INCRBY, DECR).

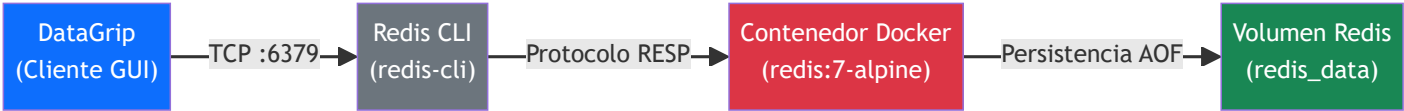
Requisitos previos

Componente	Versión mínima	Comprobación
Docker Desktop	4.x	<code>docker --version</code>
JetBrains DataGrip	2024.x	Abrir la aplicación
Redis (imagen Docker)	7.x	<code>docker pull redis:7-alpine</code>

Nota: Asegúrate de que Docker Desktop está en ejecución antes de comenzar. Si usas Windows, verifica que la integración con WSL 2 está activada en Settings → General → Use the WSL 2 based engine.

Arquitectura del laboratorio

El siguiente diagrama muestra los componentes que intervienen en este laboratorio y cómo se conectan entre sí:



Explicación: DataGrip se conecta al contenedor Redis a través del puerto 6379 mapeado al host. El contenedor almacena datos en un volumen persistente para sobrevivir reinicios.

Paso 1: Crear docker-compose.yml

Crea un fichero llamado `docker-compose.yml` en el directorio de trabajo con el siguiente contenido:

`docker-compose.yml`

```
# Servicios que componen nuestra infraestructura Redis
services:
  # Servicio principal: servidor Redis
  redis:
    # Imagen oficial de Redis 7 basada en Alpine (ligera, ~30 MB)
    image: redis:7-alpine

    # Nombre asignado al contenedor para facilitar su referencia
    container_name: redis

    # Mapeo de puertos: host:contenedor
    # El puerto 6379 del host se redirige al 6379 del contenedor
    ports:
      - "6379:6379"

    # Volúmenes persistentes para guardar datos entre reinicios
    volumes:
      - redis_data:/data

    # Comando personalizado para habilitar persistencia AOF
    # appendonly yes garantiza que las escrituras se registren en un log
    command: redis-server --appendonly yes

    # Política de reinicio: solo si el contenedor se detiene manualmente
    restart: unless-stopped

# Definición de volúmenes nombrados
volumes:
  # Volumen gestionado por Docker para persistencia de datos
  redis_data:
```

¿Por qué Alpine? La imagen `redis:7-alpine` utiliza Alpine Linux como base, reduciendo el tamaño del contenedor a ~30 MB frente a los ~100 MB de la imagen completa. Es ideal para entornos de desarrollo.

Paso 2: Levantar el contenedor

Abre una terminal en el directorio donde creaste el fichero `docker-compose.yml` y ejecuta:

Terminal

```
# Levantar el contenedor en modo background (detach)
docker-compose up -d
```

Salida esperada:

Salida

```
[+] Running 2/2
✓ Network arquitectura-datos_default Created 0.1s
✓ Container redis Started 0.3s
```

Explicación: El flag `-d` (detach) ejecuta el contenedor en segundo plano. Docker crea automáticamente una red por defecto para la comunicación entre contenedores si fuese necesario.

Paso 3: Verificar el contenedor

Comprueba que el contenedor está ejecutándose correctamente:

Terminal

```
# Listar contenedores en ejecución
docker ps
```

Salida esperada:

Salida

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
a1b2c3d4e5f6	redis:7-alpine	"docker-entrypoint.s..."	10 seconds ago	Up 9 seconds	0.0.0

Campos importantes de la salida:

- IMAGE:** La imagen utilizada (redis:7-alpine).
- STATUS:** Debe mostrar Up X seconds.
- PORTS:** Indica el mapeo 0.0.0.0:6379->6379/tcp.
- NAMES:** El nombre asignado al contenedor (redis).

Paso 4: Conectar con Redis CLI

Accede a la consola interactiva de Redis dentro del contenedor:

Terminal – Conexión interactiva

```
# Ejecutar redis-cli dentro del contenedor "redis"
docker exec -it redis redis-cli
```

Una vez dentro de la consola Redis, verifica la conectividad con el comando PING:

Redis CLI – Prueba de conexión

```
# Verificar que el servidor Redis responde
127.0.0.1:6379> PING
```

Salida esperada:

Salida

```
PONG
```

¿Qué significa PONG? La respuesta PONG confirma que el servidor Redis está activo y procesando comandos. Es el equivalente a un "estoy aquí" — como cuando haces un ping a un servidor para verificar conectividad.

Para salir de la consola Redis:

Redis CLI – Salir

```
# Salir de la consola Redis
127.0.0.1:6379> EXIT
```

Paso 5: Crear conexión en DataGrip

DataGrip permite conectarse a Redis mediante un plugin o directamente por TCP. Sigue estos pasos:

1. Abre **JetBrains DataGrip** y ve a `File` → `Data Sources` (o haz clic en el icono de base de datos en la barra lateral derecha).
2. Haz clic en el botón `+` y selecciona `Data Source` → `Redis`.
Si Redis no aparece en la lista, instala el plugin de Redis desde `Settings` → `Plugins` → `Marketplace`.
3. En la ventana de configuración, establece los siguientes parámetros:
 - **Host:** `localhost`
 - **Port:** `6379`
 - **Database:** `0` (la base de datos por defecto)
 - **Password:** *vacío* (no establecimos contraseña)
4. Haz clic en `Test Connection`. Deberías ver el mensaje `Connection successful`.
5. Haz clic en `OK` para guardar la conexión.

Resultado esperado: En el panel de DataGrip aparecerá una nueva conexión llamada `localhost` (o el nombre que asignes) con la base de datos `0` visible en el árbol de navegación.

Consejo: Si prefieres trabajar sin plugin, también puedes usar la consola de DataGrip para ejecutar comandos Redis directamente si el plugin está instalado. Asegúrate de tener el plugin habilitado en `Settings` → `Plugins` → `Installed`.

Paso 6: CRUD con DataGrip

A continuación se muestran las operaciones fundamentales de Redis con sus comandos y resultados esperados. Puedes ejecutar cada una en la consola de DataGrip o en `redis-cli`.

6.1 CREATE — Crear claves con SET

Comando SET — Crear una clave

```
# Almacena el valor "Juan García" en la clave user:1000
# Si la clave ya existe, se sobrescribe
SET user:1000 "Juan García"
```

Salida esperada:

Resultado

```
OK
```

6.2 READ — Leer claves con GET

Comando GET — Leer una clave

```
# Obtiene el valor almacenado en la clave user:1000
GET user:1000
```

Salida esperada:

Resultado

```
"Juan García"
```

6.3 CREATE — Crear múltiples claves con MSET

Comando MSET — Crear varias claves de una vez

```
# Almacena múltiples pares clave-valor en una sola operación
# Más eficiente que múltiples SET individuales (1 roundtrip vs 2)
MSET user:1001 "Ana" user:1002 "Pedro"
```

Salida esperada:

Resultado

```
OK
```

6.4 READ — Leer múltiples claves con MGET

Comando MGET — Leer varias claves

```
# Obtiene los valores de múltiples claves en una sola operación
# Devuelve una lista con los valores (nil si una clave no existe)
MGET user:1000 user:1001 user:1002
```

Salida esperada:

Resultado

```
"Juan García" → "Ana" → "Pedro"
```

6.5 DELETE — Eliminar claves con DEL

Comando DEL — Eliminar una clave

```
# Elimina la clave user:1002 del servidor
# Devuelve el número de claves eliminadas (0 si no existía)
DEL user:1002
```

Salida esperada:

Resultado

```
(integer) 1
```

6.6 READ — Verificar existencia con EXISTS

Comando EXISTS – Comprobar si una clave existe

```
# Verificar si una clave existe (1 = existe, 0 = no existe)
EXISTS user:1000
```

Salida esperada:

Resultado

```
(integer) 1
```

Comando EXISTS – Clave inexistente

```
# Verificar una clave que no existe
EXISTS user:9999
```

Salida esperada:

Resultado

```
(integer) 0
```

Resumen CRUD:

Operación	Comando	Resultado
Crear	SET user:1000 "Juan García"	OK
Leer	GET user:1000	"Juan García"
Crear varios	MSET user:1001 "Ana" user:1002 "Pedro"	OK
Leer varios	MGET user:1000 user:1001 user:1002	["Juan García", "Ana", "Pedro"]
Eliminar	DEL user:1002	(integer) 1
Existe	EXISTS user:1000	(integer) 1
No existe	EXISTS user:9999	(integer) 0

Paso 7: TTL y expiración

Redis permite establecer un tiempo de vida (TTL) a las claves. Esto es útil para datos temporales como sesiones, tokens o cachés.

7.1 SETEX — Crear clave con expiración

Comando SETEX — Crear con TTL de 60 segundos

```
# Almacena la clave session:abc con TTL de 60 segundos
# SETEX = SET + EXPIRE en una sola operación atómica
SETEX session:abc 60 "userId:1000"
```

Salida esperada:

Resultado

```
OK
```

7.2 EXPIRE — Establecer expiración a una clave existente

Comando EXPIRE — Añadir TTL a una clave existente

```
# Establece expiración de 300 segundos (5 minutos) a user:1000
EXPIRE user:1000 300
```

Salida esperada:

Resultado

```
(integer) 1
```

7.3 TTL — Consultar tiempo restante

Comando TTL — Consultar segundos restantes

```
# Consultar el TTL de una clave
# -2 = la clave no existe
# -1 = la clave no tiene expiración
# >0 = segundos restantes
TTL user:1000
```

Salida esperada: (el valor variará según cuándo lo ejecutes)

Resultado

```
(integer) 295
```

Valores de TTL:

- -2: La clave no existe.
- -1: La clave existe pero no tiene expiración.
- > 0: Segundos restantes antes de que la clave expire.

7.4 PERSIST — Eliminar expiración

Comando PERSIST — Quitar TTL

```
# Eliminar la expiración de una clave
# La clave ya no caducará automáticamente
PERSIST session:abc
```

Salida esperada:

Resultado

```
(integer) 1
```

Paso 8: Contadores con INCR

Las operaciones de incremento y decremento son **atómicas** en Redis. Esto garantiza que múltiples clientes concurrentes nunca obtienen resultados inconsistentes, algo fundamental para contadores distribuidos.

8.1 INCR — Incrementar en 1

Comando INCR — Incrementar contador

```
# Inicializar un contador en 0
SET page:views 0

# Incrementar el contador en 1
INCR page:views
```

Salida esperada:

Resultado

```
(integer) 1
```

Comando INCR — Incrementar nuevamente

```
# Incrementar una segunda vez
INCR page:views
```

Salida esperada:

Resultado

```
(integer) 2
```

8.2 INCRBY — Incrementar en una cantidad

Comando INCRBY — Incrementar en 5

```
# Incrementar el contador en 5 unidades
INCRBY page:views 5
```

Salida esperada:

Resultado

```
(integer) 7
```

8.3 DECR — Decrementar en 1

Comando DECR — Decrementar contador

```
# Decrementar el contador en 1 unidad  
DECR page:views
```

Salida esperada:

Resultado

```
(integer) 6
```

¿Por qué son atómicas? Redis ejecuta todos los comandos en un único hilo. Cuando ejecuta INCR, nadie más puede ejecutar comandos hasta que termine. Esto garantiza que dos clientes concurrentes nunca obtienen el mismo resultado. Es la misma propiedad que utilizan los bancos para contadores de transacciones.

Caso real: GitHub usa INCR para rate limiting de su API. Cada usuario autenticado tiene derecho a 5.000 peticiones por hora. Redis almacena un contador atómico por usuario que se incrementa con cada petición y se resetea automáticamente tras una hora con EXPIRE.

Preguntas de reflexión

Antes de continuar con el siguiente laboratorio, reflexiona sobre las siguientes preguntas:

1. ¿Por qué Redis es más rápido que una base de datos relacional?

Piensa en dónde se almacenan los datos (RAM vs disco) y en la complejidad de las operaciones ($O(1)$ vs consultas SQL con JOINS).

2. ¿Cuándo usarías Redis en lugar de MariaDB?

Considera los casos de uso: caché de consultas, sesiones, contadores, colas de mensajes. ¿Cuáles se adaptan mejor a Redis y cuáles necesitan una BD relacional?

3. ¿Qué pasaría si no configuramos appendonly yes?

Si solo usamos RDB (snapshots), ¿qué datos se perderían al reiniciar el contenedor? ¿Cuánto tiempo de datos podrías perder?

4. ¿Cuál es la diferencia entre SET y SETEX?

¿En qué situaciones usarías uno u otro? Piensa en el caso de las sesiones de usuario vs. datos permanentes.

5. ¿Por qué INCR es útil para contadores distribuidos?

Imagina una aplicación con múltiples servidores. Si cada servidor tuviera su propio contador local, ¿qué problema surgiría? ¿Cómo resuelve Redis la concurrencia?

Comandos de referencia rápida

Comando	Descripción	Ejemplo
PING	Verificar conectividad	PING → PONG
SET	Crear/actualizar una clave	SET clave valor
GET	Leer el valor de una clave	GET clave
SETEX	Crear clave con expiración	SETEX clave 60 valor
MSET	Crear múltiples claves	MSET k1 v1 k2 v2
MGET	Leer múltiples claves	MGET k1 k2 k3
DEL	Eliminar claves	DEL clave
EXISTS	Verificar existencia	EXISTS clave
EXPIRE	Establecer TTL	EXPIRE clave 300
TTL	Consultar TTL restante	TTL clave
INCR	Incrementar en 1	INCR contador
DECR	Decrementar en 1	DECR contador
INCRBY	Incrementar en N	INCRBY contador 5



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

[Human Language and Accessibility Technologies Group \(HULAT\)](#)

[Departamento de Informática](#)

[Universidad Carlos III de Madrid](#)